

FAREI TICKETING SYSTEM

1. Analysis

1.1 Description:

The system takes in requests about faulty equipment from the user and stores it in a database. This is passed to that department's supervisor for approval and if accepted, the registry documents the movement of the equipment. The IT department will then try to repair and if successful the technician passes it to the ITO for approval. If the equipment needs a new component technician can send a request to the ITO for approval. ITO can also send it to a third party if necessary. The loop only closes when the user sends feedback about the equipment.

1.2 Languages and platform:

We are using a mix of HTML, CSS and C# to create the pages and its functionality. We are using SQL to store every information into a database. The platform that we are using is visual studio due to it generating most of our pages instead of building it from scratch and ASP.NET MVC (Model-View-Controller) for web development framework.

2. Wireframe

We have also designed some wireframes(as shown below)and built the system accordingly.

Login page



Food and Agricultural Research
and Extension Institute

Email

password

Login

Forgot your email or password?

Signup page



Food and Agricultural Research
and Extension Institute

Email

password

re-enter password

Choose role ▾

sign up

User page

Test



Farm and Agricultural Research
and Extension Institute

 LOG OUT

Welcome -----

HOME

REQUEST STATUS

DEMO

Request Form Application



Farm and Agricultural Research
and Extension Institute

 LOG OUT

Welcome -----

HOME

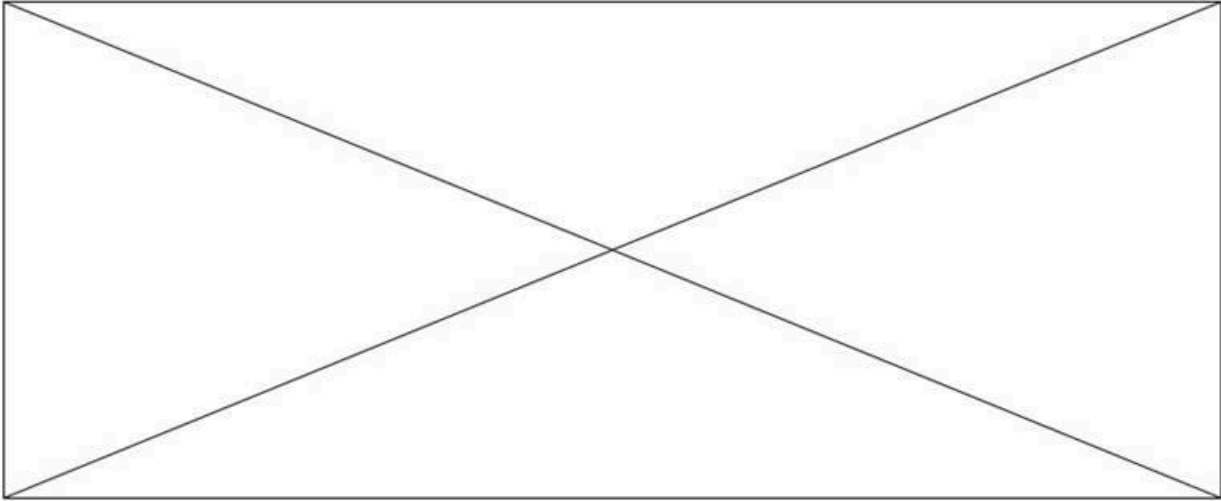
REQUEST STATUS

DEMO

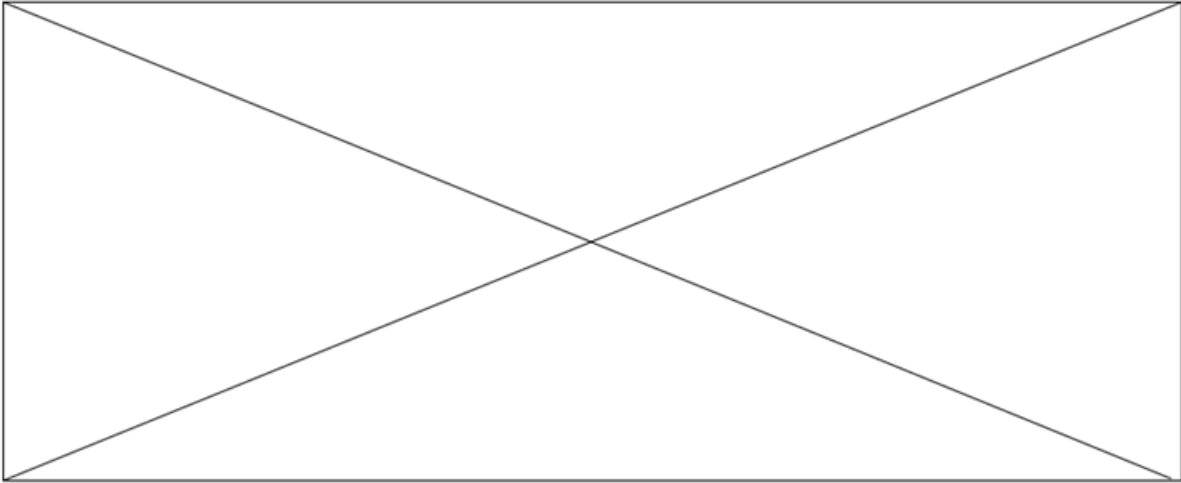
Requests Issued

NO.	ITEM NAME	ISSUE	DATE SUBMITTED	DETAILS	STATUS
					

Supervisor page

	Food and Agricultural Research and Extension Institute	logout
Home		
Approved request		Search user....
		

IT Officer page

 Food and Agricultural Research and Extension Institute		Logout
Home		
Form request		Search user....
		

3. Front end & back end

-Login page

When the user first enters the website it will display the welcome page. They will have a link that sends them to the login page. (The “register” will be removed when the website is launched and will be only available to the admin.)

Welcome

[Please login first.](#)

Log in

Use a local account to log in.

☐ Remember me?

```

@{
    ViewData["Title"] = "Index";
    var username = User.Identity.Name;
    var Type = "default";

    @if (username == null)
    {
        Type = "default";
    }
    else
    {
        foreach (var claim in User.Claims)
        {
            Console.WriteLine($"Type: {claim.Type}, Value: {claim.Value}");
        }
        @foreach (var User in Model.AllUsers)
        {
            if (username.Equals(User.UserName))
            {
                Type = User.Type;
                break;
            }
        }
    }
}

@if (Type.Equals("default"))
{
    <div class="text-center">
        <h1 class="display-4">Welcome</h1>
        <p>
            <a asp-area="Identity" asp-page="/Account/Login">
                Please login first.
            </a>
        </p>
    </div>
}

```

This verify if the user is logged in or not.

It will display this if not logged in

It will display different pages according to the user type. The user types are:

- User
- Supervisor
- Registry
- Technician
- ITO
- Admin

The user does not need to input his/her user type as during the registry they have already been assigned one.

Here are some different types of pages that will be display:

User page:

FAREI

Request

My request

Hello KEVISH2@gmail.com! Logout

User

[Create New](#)

RequestDate	ResponsibleOfficer	EquipmentType	ProblemDescription	
6/24/2025	KEVISH2@gmail.com	printer	ink problem	Details
6/24/2025	KEVISH2@gmail.com	Monitor	displaying white bars on the screen	Details
6/24/2025	KEVISH4@gmail.com	CPU	Overheating	Details

```
else if (Type.Equals("User"))
{
    <h1>User</h1>
    <p>
        <a asp-action="Create">Create New</a>
    </p>
    <table class="table">
        <thead>
            <tr>
                <th>RequestDate</th>
                <th>ResponsibleOfficer</th>
                <th>EquipmentType</th>
                <th>ProblemDescription</th>
                <th></th>
            </tr>
        </thead>
        <tbody>
            @foreach (var item in Model.FormReqDb)
            {
                <tr>
                    <td>@Html.DisplayFor(modelItem => item.RequestDate)</td>
                    <td>@Html.DisplayFor(modelItem => item.ResponsibleOfficer)</td>
                    <td>@Html.DisplayFor(modelItem => item.EquipmentType)</td>
                    <td>@Html.DisplayFor(modelItem => item.ProblemDescription)</td>
                    <td><a asp-action="Details" asp-route-id="@item.Id">Details</a></td>
                </tr>
            }
        </tbody>
    </table>
```

Supervisor page:

FAREI

Request

Hello KEVISH1@gmail.com! Logout

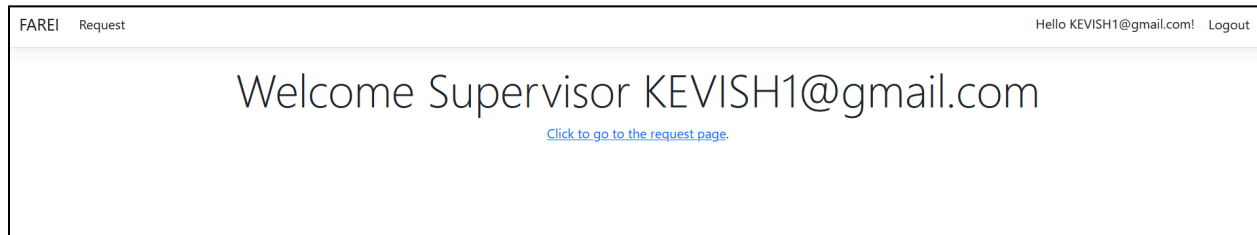
Welcome Supervisor KEVISH1@gmail.com

[Click to go to the request page.](#)

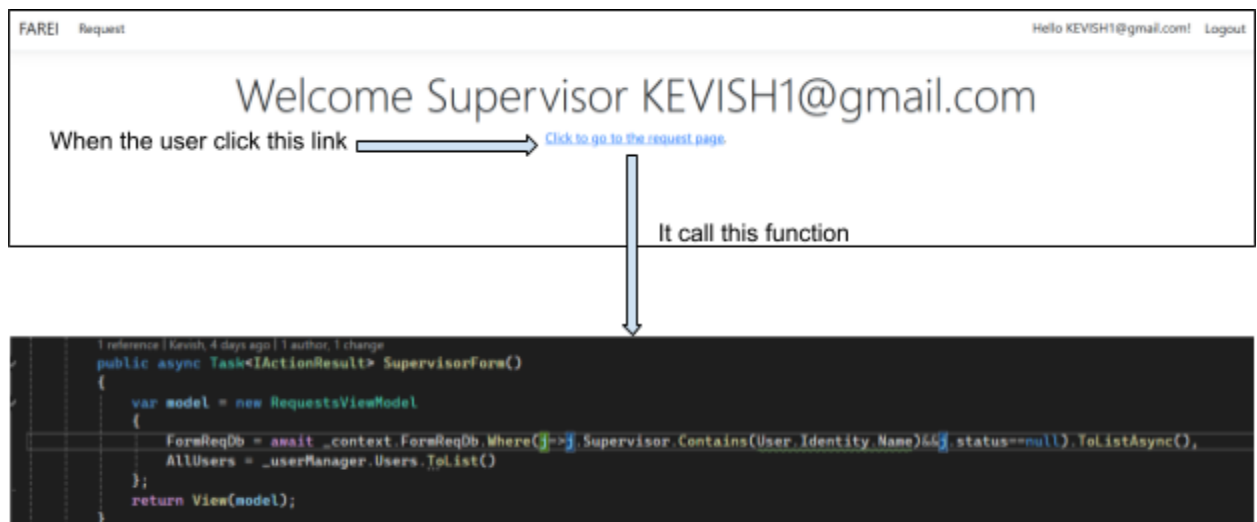
```
else if (Type.Equals("Supervisor"))
{
    <div class="text-center">
        <h1 class="display-4">Welcome Supervisor @username</h1>
        <p> <a asp-controller="FormReqDb" asp-action="SupervisorForm">Click to go to the request page</a>.</p>
    </div>
```


-Supervisor page

When a supervisor logs in on his/her account, it will not display the request page immediately. It will simply display a welcome page.



The reason for that is because it needs to call a function to filter out the request. If it simply goes to the .cshtml it will display all of the requests all at one.



After that the function will create a class RequestViewModel and it contains the FormReqDb, it will have all of the requests where the supervisor name is equal to the current login supervisor and where the status is equal null and also it will return a list of AllUsers. Then after that it will return the view(model) which will create a Supervisor form page with the model which has all the necessary information.

Supervisor

RequestDate	ResponsibleOfficer	EquipmentType	ProblemDescription	
6/24/2025	KEVISH2@gmail.com	printer	ink problem	Details
6/24/2025	KEVISH2@gmail.com	Monitor	displaying white bars on the screen	Details
6/24/2025	KEVISH4@gmail.com	CPU	Overheating	Details

```
@using FAREI_Project.ViewModel
@model RequestsViewModel
<h1>Supervisor</h1>
<table class="table">
    <thead>
        <tr>
            <th>RequestDate</th>
            <th>Responsible Officer</th>
            <th>Equipment Type</th>
            <th>ProblemDescription</th>
            <th></th>
        </tr>
    </thead>
    <tbody>
        @foreach (var item in Model.FormReqDb) {
            <tr>
                <td>@Html.DisplayFor(modelItem => item.RequestDate)</td>
                <td>@Html.DisplayFor(modelItem => item.ResponsibleOfficer)</td>
                <td>@Html.DisplayFor(modelItem => item.EquipmentType)</td>
                <td>@Html.DisplayFor(modelItem => item.ProblemDescription)</td>
                <td><a asp-action="DetailsSupervisorForm" asp-route-id="@item.Id">Details</a></td>
            </tr>
        }
    </tbody>
</table>
```

In order for the supervisor to accept or reject a request they will have to first click on detail. It will display all of the necessary details of the request by fetching it using its ID.

FormReqDb

RequestDate	6/24/2025
Site	St Pierre
Department	HR Dept
ResponsibleOfficer	KEVISH2@gmail.com
ContactPhone	5678899
EquipmentType	Monitor
ProblemDescription	displaying white bars on the screen
SerialNumber	3oempd2
status	

[Back to List](#)

```
<table>
<tr>
<td>FormReqDbs.</td>
<td>
<tr>
<tr>
<td class="row">
<td class="col-sm-2">@Html.DisplayNameFor(model => model.FormReqDbs.RequestDate)</td>
<td class="col-sm-18">@Html.DisplayFor(model => model.FormReqDbs.RequestDate)</td>
<td class="col-sm-2">@Html.DisplayNameFor(model => model.FormReqDbs.Site)</td>
<td class="col-sm-18">@Html.DisplayFor(model => model.FormReqDbs.Site)</td>
<td class="col-sm-2">@Html.DisplayNameFor(model => model.FormReqDbs.Department)</td>
<td class="col-sm-18">@Html.DisplayFor(model => model.FormReqDbs.Department)</td>
<td class="col-sm-2">@Html.DisplayNameFor(model => model.FormReqDbs.ResponsibleOfficer)</td>
<td class="col-sm-18">@Html.DisplayFor(model => model.FormReqDbs.ResponsibleOfficer)</td>
<td class="col-sm-2">@Html.DisplayNameFor(model => model.FormReqDbs.ContactPhone)</td>
<td class="col-sm-18">@Html.DisplayFor(model => model.FormReqDbs.ContactPhone)</td>
<td class="col-sm-2">@Html.DisplayNameFor(model => model.FormReqDbs.EquipmentType)</td>
<td class="col-sm-18">@Html.DisplayFor(model => model.FormReqDbs.EquipmentType)</td>
<td class="col-sm-2">@Html.DisplayNameFor(model => model.FormReqDbs.ProblemDescription)</td>
<td class="col-sm-18">@Html.DisplayFor(model => model.FormReqDbs.ProblemDescription)</td>
<td class="col-sm-2">@Html.DisplayNameFor(model => model.FormReqDbs.SerialNumber)</td>
<td class="col-sm-18">@Html.DisplayFor(model => model.FormReqDbs.SerialNumber)</td>
<td class="col-sm-2">@Html.DisplayNameFor(model => model.FormReqDbs.status)</td>
<td class="col-sm-18">@Html.DisplayFor(model => model.FormReqDbs.status)</td>
</td>
</tr>
</div>
</div>
```

After reviewing the supervisor can either click on accept or reject and it will update the status in the database.

```
<div class="form-group">

  <form asp-action="ChangeStatus" method="post" class="btn btn-primary">
    <input type="hidden" name="id" value="@Model.FormReqDbs.Id" />
    <input type="hidden" name="Accepted" value="Accepted"></input>
    <input type="submit" value="Accept" class="btn btn-primary" />
  </form>

  <form asp-action="ChangeStatus" method="post" class="btn btn-secondary">
    <input type="hidden" name="id" value="@Model.FormReqDbs.Id" />
    <input type="hidden" name="Accepted" value="reject"></input>
    <input type="submit" value="reject" class="btn btn-secondary" />
  </form>

</div>
```

After clicking one of the buttons

```
public async Task<IActionResult> ChangeStatus(int id, String Accepted)
{
    var formReqDb = await _context.FormReqDb.FindAsync(id);
    if (formReqDb == null)
    {
        return NotFound();
    }
    formReqDb.status = Accepted;
    try
    {
        await _context.SaveChangesAsync();
    }
    catch (DbUpdateConcurrencyException)
    {
        if (!FormReqDbExists(id))
        {
            return NotFound();
        }
        else
        {
            throw;
        }
    }
    return RedirectToAction(nameof(SupervisorForm));
}
```

It will call this function

Then after that it redirects to the Supervisor page.

-Form:

After a successful login,user gets directed to the Repair/Service Request Form

The screenshot shows a web browser window with the address bar displaying 'localhost:7047/FormReqDb/Create'. The page has a navigation bar with 'Home' and 'Privacy' links. The main heading is 'Repair/Service Request Form'. Below this is the 'Request Details' section, which includes four input fields: 'Date' (with the value '01/01/0001' and a calendar icon), 'Site', 'Department' (a dropdown menu showing '-- Select Department --'), and 'Responsible Officer'. Below these is a 'Contact Phone' input field. The 'Equipment Section' follows, featuring a table with three columns: 'Equipment Type', 'Serial Number', and 'Problem'. Each column has an empty input field. Below the table is an 'Add Equipment' button. At the bottom of the form are 'Submit' and 'Cancel' buttons.

Create.cshtml (New Form Display)

```
@model FormRequest.Models.FormReqDb
@{
    ViewData["Title"] = "Repair/Service Request Form";
}
<h2>Repair/Service Request Form</h2>
<hr />
<form asp-action="Create">
    <h4>Request Details</h4>
    <div class="row mb-3">
        <div class="col-md-3">
            <label asp-for="RequestDate">Date:</label>
            <input asp-for="RequestDate" type="date" class="form-control" />
        </div>
        <div class="col-md-3">
            <label asp-for="Site">Site:</label>
            <input asp-for="Site" class="form-control" />
        </div>
        <div class="col-md-3">
            <label asp-for="Department">Department:</label>
            <select asp-for="Department" class="form-control">
                <option>-- Select Department --</option>
                <option>IT Dept</option>
                <option>Finance Dept</option>
                <option>HR Dept</option>
            </select>
        </div>
        <div class="col-md-3">
            <label asp-for="ResponsibleOfficer">Responsible Officer:</label>
            <input type="text" class="form-control" />
        </div>
    </div>
    <div class="form-group">
        <input type="text" class="form-control" />
    </div>
    <div class="form-group">
        <input type="button" value="Add Equipment" />
    </div>
    <div class="form-group">
        <input type="button" value="Submit" />
        <input type="button" value="Cancel" />
    </div>
</form>
```

Equipment Section:(Dynamic list to enable user to select multiple equipment in case more than one needs to be repaired/serviced)+(Equipment details retrieved through an inventory table where SerialNumber=PK)

Still in progress

```
<thead class="table-light">
  <tr>
    <th>Equipment Type</th>
    <th>Serial Number</th>
    <th>Problem</th>
  </tr>
</thead>
<tbody>
  @for (int i = 0; i < Model.Equipments.Count; i++)
  {
    <tr>
      <td>
        <input asp-for="Equipments[@i].EquipmentType" class="form-control" />
        <input type="hidden" asp-for="Equipments[@i].Id" />
      </td>
      <td>
        <input asp-for="Equipments[@i].SerialNumber" class="form-control" />
      </td>
      <td>
        <input asp-for="Equipments[@i].ProblemDescription" class="form-control" />
      </td>
    </tr>
  }
</tbody>
</table>
<button type="button" class="btn btn-secondary mb-3">Add Equipment</button>
```

Registry Section: :(Dynamic list to enable Registry to record multiple movements(In case there is more than one.))

Still in progress

```
<thead class="table-light">
  <tr>
    <th>From</th>
    <th>To</th>
    <th>Date</th>
    <th>Remarks</th>
    <th>Approved</th>
    <th>Invalid</th>
  </tr>
</thead>
<tbody>
  @for (int i = 0; i < Model.Registries.Count; i++)
  {
    <tr>
      <td>
        <input asp-for="Registries[@i].From" class="form-control" />
        <input type="hidden" asp-for="Registries[@i].Id" />
      </td>
      <td><input asp-for="Registries[@i].To" class="form-control" /></td>
      <td><input asp-for="Registries[@i].MovementDate" type="date" class="form-control" /></td>
      <td><textarea asp-for="Registries[@i].Remarks" class="form-control"></textarea></td>
      <td class="text-center">@Html.CheckBoxFor(m => m.Registries[i].IsApproved)</td>
      <td class="text-center">@Html.CheckBoxFor(m => m.Registries[i].IsInvalid)</td>
    </tr>
  }
</tbody>
</table>
<button type="button" class="btn btn-secondary mb-3">Add Registry Entry</button>
```

Registry Page:

The registry(Entity) has the purpose of tracking movement of the equipment. The registry entity logs in and a registry page is displayed where he will be able to view 3 tabs:

- 1)All Requests:Displays all submitted requests approved by the Supervisor.
- 2)On-Site:Displays approved requests where repair will be dealt with On-site by IT Technician.
- 3)In-Transit:Displays approved requests where IT Technician will have to move to perform reparation/servicing.

The screenshot displays the 'Registry Requests' page in a web browser. The page has a navigation bar with three tabs: 'All Requests' (selected), 'On-Site', and 'In Transit'. Below the tabs is a table with the following columns: Date, Officer, Equipment, Problem, and Details. The table contains three rows of data. A yellow box labeled 'Navigation links/buttons' points to the top navigation bar. A yellow box labeled 'Display list requests' points to the table. A yellow box labeled 'Details button for each request record' points to the 'Details' column of the table. Two code snippets are shown: one for the controller action 'Registry' and another for the view's foreach loop rendering the table rows.

```
<a asp-action="Registry" class="btn btn-secondary">All Requests</a>
<a asp-action="OnSite" class="btn btn-success">On-Site</a>
<a asp-action="InTransit" class="btn btn-warning">In Transit</a>
</div>
```

Registry Requests

All Requests On-Site In Transit

Date	Officer	Equipment	Problem	Details
21/06/2025	Kp	Laptop	Fan Overheating	Details
22/06/2025	v.k	Laptop	dddddd	Details
22/06/2025	sa	Laptop	Fan Overheating	Details

```
public IActionResult Registry()
{
    var requests = _context.FormReqDb.ToList();
    return View(requests);
}
```

```
@foreach (var item in Model)
{
    <tr>
        <td>@item.RequestDate.ToShortDateString()</td>
        <td>@item.ResponsibleOfficer</td>
        <td>@item.EquipmentType</td>
        <td>@item.ProblemDescription</td>
        <td>
            <a asp-action="RegistryDetails"
               asp-route-id="@item.Id" class="
               btn btn-outline-primary btn-sm">
                Details
            </a>
        </td>
    </tr>
}
```

Registry Details Page:

Clicking the Details button opens the Registry Details Page which shows detailed request form and at the bottom there is a Registry Form Section, where Registry has to fill in required details to record Movement of Equipment and also Validates the movement taking [place. In](#) this form, Registry will also classify if the repair will be dealt with On-Site/In-Transit

Details

FormReqDb

RequestDate	21/06/2025
Site	Reduit
Department	Finance Dept
ResponsibleOfficer	K.p
ContactPhone	58081279
EquipmentType	Laptop
ProblemDescription	Fan Overheating
SerialNumber	B123d

Add Registry Entry

From

To

MovementDate

01/01/0001

Remarks

☐ IsValid

☐ IsOnSite

☐ IsInTransit

```
public IActionResult RegistryDetails(int id)
{
    var formReq = _context.FormReqDb
        .Include(f => f.Registries)
        .FirstOrDefault(x => x.Id == id);

    if (formReq == null) return NotFound();

    var viewModel = new RegistryViewModel
    {
        FormReqDb = formReq,
        Registry = new Registry
        {
            FormReqDbId = id,
        },
        RegistryList = formReq.Registries.ToList()
    };
    return View(viewModel);
}
```

Displays the detailed page for a specific request.

```
<h3>Add Registry Entry</h3>
<form asp-action="SubmitRegistry" method="post">
    @Html.AntiForgeryToken()

    <input asp-for="Registry.FormReqDbId" type="hidden" />

    <div>
        <label asp-for="Registry.From"></label>
        <input asp-for="Registry.From" class="form-control" />
        <span asp-validation-for="Registry.From" class="text-danger"></span>
    </div>

    <div>
        <label asp-for="Registry.To"></label>
        <input asp-for="Registry.To" class="form-control" />
        <span asp-validation-for="Registry.To" class="text-danger"></span>
    </div>

    <div>
```

To submit Registry Form Entries

Form to add Registry Inputs

```
[HttpPost]
[ValidateAntiForgeryToken]
0 references
public IActionResult SubmitRegistry(RegistryViewModel model)
{
    if (!ModelState.IsValid)
    {
        model.FormReqDb = _context.FormReqDb
            .Include(f => f.Registries)
            .FirstOrDefault(f => f.Id == model.Registry.FormReqDbId);
        model.RegistryList = model.FormReqDb.Registries.ToList();

        return View("RegistryDetails", model);
    }
    _context.Registry.Add(model.Registry);
    _context.SaveChanges();
    return RedirectToAction("Registry");
}
```

Handles submission of Registry Entry from Registry Details Page

On-Site & In Transit

Registry Requests

All Requests On-Site In Transit

Date	Officer	Equipment	Problem	Details
21/06/2025	K.p	Laptop	Fan Overheating	Details
22/06/2025	v.k	Laptop	dddddd	Details
22/06/2025			Fan Overheating	Details

```
public IActionResult OnSite()
{
    var forms = _context.FormReqDb
        .Include(f => f.Registries)
        .Where(f => f.Registries.Any(r => r.IsOnSite))
        .ToList();
    return View("Registry", forms);
}
```

Filters Requests where IsOnSite=True in the Registry Form

Registry Requests

All Requests On-Site In Transit

Date	Officer	Equipment
22/06/2025	v.k	Laptop

Suggestion?

On-Site and In-Transit's Details button will display all the information about the form request + Registry Form inputs and will have a button "Approve".

For example: If request is under "On-Transit" where the IT Tech will have to displace to make repairs. That site's Registry will Log in, go under In-Transit tab, click required request from the list-click on details and click on approve button, confirming that the IT-Tech came for that particular request / movement is valid.

In Progress

Adding a search functionality in the Registry Page where Registry can search requests according to the Responsible Officer' name.

